Day 2

March - 01 - 2026

Modern Async Pattern

21- Closures.JS

```
function init (){
    let name = "mozilla";
    function displayName (){
        console.log (name);
    }
    displayName ();
}
init (); // mozilla
console.log (name);
        ↳ name not defined
```

```
function makeFun () {
    const name = "mozilla";
    function displayName (){
        console.log (name);
    }
    return displayName;
}
let myFunc = makeFunc ();
myFunc ();
```

इस function का इतना ही code आना चाहिए और name displayname के अंदर नहीं है तो name undefine आना चाहिए पर mozilla आ रहा है।

[illegible]

[illegible]

→ जितने भी function बनाये हो उनके लिए Closure
बनता है Closure is tiny

→ Closure जिस function का उसी function उस
Closure को return कर रहा है

→ माने कि एक function एक object लौटाते वक्त से बना है

```
function [illegible]() {
    function Go(name) {
        return 'Name of your Company is ${name}'
    }
    return Go
}
const [illegible] = [illegible]()
const myCompanyName = [illegible]("Zomato")
```

```
function starter (guest) {
  let guestName = guest;

  function Zomato () {
    console.log(`Hi ${guestName}, from Zomato`);
  }

  function Blinkit () {
    console.log(`Hi ${guestName}, from Blinkit`);
  }

  Zomato();
  Blinkit()
}
let Aditya = starter("Aditya")
let piyush = starter("piyush");
```

↳

Hi: Aditya from Zomato
Hi: Aditya from Blinkit
Hi: piyush from Zomato
Hi: piyush from blinkit.

Note use Memo ↳ अगर function में change नहीं हुआ तो function को Run ही न करो.

```
<body>
<button id="clickBtn">Click me</button>

<script>
const buttonRef = window.document.getElementById('clickBtn')

buttonRef.remove()
</script>
```

Here is the by after buttonRef.remove() के बाद buttonRef में भरी हुई reference तो वैसा है। इसे garbage Collector ले तो ही जाएगा so we done this

buttonRef = null

↳ अब garbage Collector इसे clear कर देगा।